

ATTENZIONE! QUESTO MATERIALE DIDATTICO È PER USO PERSONALE DELLO STUDENTE
ED È COPERTO DA COPYRIGHT. NE È VIETATA LA RIPRODUZIONE O IL RIUTILIZZO ANCHE PARZIALE,
AI SENSI E PER GLI EFFETTI DELLA LEGGE SUL DIRITTO D'AUTORE.

Specifiche Logiche

Ingegneria del Software Avanzata

Università di Ferrara

CdLM in Ingegneria Informatica e dell'Automazione

A.A. 2021/2022

Marco Alberti (lbrmrc@unife.it)

Specifiche logiche

- Specificano le proprietà del programma per mezzo di *formule logiche*, cioè esprimendo le proprietà in un linguaggio formale di tipo logico.
- I linguaggi logici più usati sono:
 - logica proposizionale (formule senza variabili)
 - logica dei predicati, o del primo ordine (in cui le formule possono contenere variabili e quantificatori)
- Per l'ingegneria del software: linguaggi formali per specifiche dichiarative
- Specifiche verificabili staticamente o dinamicamente
- Qui introduciamo brevemente (e informalmente) la logica del primo ordine

Logica del primo ordine (o dei predicati)

- Un linguaggio logico si basa su un alfabeto.
- *Alfabeto* composto da:
 - insieme C dei *simboli di costante*
 - insieme V dei *simboli di variabile*
 - insieme F dei *simboli di funzione*
 - insieme P dei *simboli di predicato*
 - *connettori logici*: **and**, **or**, **not**, **implies**, $\equiv$
 - *parentesi*
 - *quantificatori*: $\forall$ (**forall**, universale), $\exists$ (**exists**, esistenziale)

Sintassi

- **Termini elementari:**
 - costanti,
 - variabili,
 - applicazioni di funzioni ($f(\dots)$) a termini elementari
- **Esempi:**
 - costante c_0
 - variabile x
 - se f e g sono simboli di funzione: $f(x)$, $g(f(c_0), x)$

Sintassi

- **Formule atomiche**: applicazione di simboli di predicato ($p(\dots)$) a termini elementari
- Esempi: se p è un simbolo di predicato,
 - $p(x)$
 - $p(x, f(c_0))$sono formule atomiche
- Alcuni predicati e funzioni hanno sintassi particolare:
 - es. $<$, notazione infissa: $x < c_0$ è una formula atomica

Sintassi

Formule ben formate (fbf):

- le formule atomiche sono fbf
- l'applicazione usuale dei connettori logici a fbf è una fbf
- se A è una fbf e X è una variabile:
 - $\forall X A$
 - $\exists X A$sono fbf

Esempi di formule ben formate:

- $p(x)$
- $p(x)$ **and** $p(f(c_0))$
- $\forall x p(x)$
- $\forall x \exists y (p(x) \text{ and } p(f(y)))$

Variabili

- Una formula senza variabili è detta *ground*.
- In una formula, una variabile è detta
 - *libera* se non è quantificata
 - *legata* altrimenti
- Una formula senza variabili libere è detta *chiusa*.
- La *chiusura* di una formula si ottiene quantificando universalmente tutte le variabili libere

Semantica

- Una *interpretazione* associa un significato ai simboli di
 - costante: elemento del dominio D
 - funzione n -aria: funzione da D^n a D
 - predicato n -ario: relazione n -aria (sottoinsieme di D^n)
- In questo modo ogni formula atomica ground assume un valore di verità, uguale a quello della sua interpretazione nel dominio del discorso
- Il valore di verità di una fbf chiusa si ottiene applicando le tabelle di verità dei connettivi logici usati ai valori di verità delle formule atomiche, e i quantificatori alle variabili legate
- Il valore di verità di una fbf con variabili libere richiede l'assegnamento a ogni variabile di un termine ground.
- Assegneremo ai simboli i significati consueti anziché definirli ogni volta

Tabelle di verità e quantificatori

A	B	not A	A or B	A and B	A implies B	A $\equiv$ B
V	V	F	V	V	V	V
V	F		V	F	F	F
F	V	V	V	F	V	F
F	F		F	F	V	V

Se φ è una formula:

- **forall** $x \varphi$ è vera se, per ogni termine ground t , è vera la formula che si ottiene sostituendo t a ogni occorrenza di x in φ
- **exists** $x \varphi$ è vera se, per almeno un termine ground t , è vera la formula che si ottiene sostituendo t a ogni occorrenza di x in φ

Esempi

Dominio: $\mathbb{N}$

1. $x > y$ **and** $y > z$ **implies** $x > z$

2. $x = y \equiv y = x$

3. **for all** x, y, z ($x > y$ **and** $y > z$ **implies** $x > z$) *(chiusura di 1)*

4. $x + 1 < x - 1$

5. $x + 1 > x - 1$

6. **for all** x (**exists** y ($y = x + z$))

7. $x > 3$ **or** $x < -6$

Logica come linguaggio di specifica

- Formule logiche rappresentano requisiti di
 - interi programmi
 - sottoprogrammi
 - frammenti di codice
- Le formule logiche possono riguardare i valori di
 - input e output
 - parametri e valori di ritorno
 - variabili
 - attributi delle strutture dati (es. dimensione di un vettore)

Specifica di programmi completi

- Precondizione: formula logica avente come variabili libere gli input del programma

$$\{\text{Pre } (i_1, i_2, \dots, i_n) \}$$

- Postcondizione: formula logica avente come variabili libere gli input e gli output del programma

$$\{\text{Post } (o_1, o_2, \dots, o_m, i_1, i_2, \dots, i_n)\}$$

Asserzioni input-output

- Dato un programma P , la *proprietà o requisito*

$$\{\text{Pre } (i_1, i_2, \dots, i_n) \}$$
$$P$$
$$\{\text{Post } (o_1, o_2, \dots, o_m, i_1, i_2, \dots, i_n)\}$$

significa che se la preconditione è vera prima dell'esecuzione del programma, la postcondizione sarà vera dopo l'esecuzione

Esempi

- $\{\text{exists } z (i_1 = z * i_2)\}$
P
 $\{o_1 = i_1 / i_2\}$
- $\{i_1 > i_2\}$
P
 $\{i_1 = i_2 * o_1 + o_2 \text{ and } o_2 \geq 0 \text{ and } o_2 < i_2\}$
- $\{\text{true}\}$
P
 $\{o = i_1 \text{ or } o = i_2\} \text{ and } o \geq i_1 \text{ and } o \geq i_2\}$

Esempi

- $\{n > 0\}$

P

$$\{o = \sum_{k=1..n} i_k\}$$

- $\{n > 0\}$

P

$$\{\textbf{for all } i (1 \leq i \leq n) \textbf{ implies } (o_i = i_{n-i+1})\}$$

Esempi

$\{i_1 > 0 \text{ and } i_2 > 0\}$

P

$\{(\text{exists } z_1, z_2 (i_1 = o * z_1 \text{ and } i_2 = o * z_2)$

$\text{and not (exists } h (\text{exists } z_1, z_2 (i_1 = h * z_1 \text{ and } i_2 = h * z_2) \text{ and } h > o)))\}$

Specifica di frammenti di programma

- Si possono specificare precondizioni e postcondizioni relativamente ai parametri di ingresso e di uscita di una procedura

- $\{n > 0\}$
procedure search (table: in integer_array; n: in integer; element: in integer; found: out Boolean);
 $\{found \equiv (\textbf{exists } i (1 \leq i \leq n \textbf{ and } table(i) = element))\}$

Specifica di frammenti di programma

- *Asserzioni intermedie*: si riferiscono alle variabili del programma, piuttosto che ai valori di I/O

- $\{n > 0\}$

codice che assegna true a found se esiste un elemento dell'array table di n elementi uguale a element, false altrimenti

$\{found \equiv (\textbf{exists } i (1 \leq i \leq n \textbf{ and } table(i) = \text{element}))\}$

Asserzioni intermedie

- $\{n > 0\}$
codice che inverte l'ordine degli n elementi dell'array a
{for all i ($1 \leq i \leq n$) implies ($a(i) = \text{old_}a(n - i + 1)$)}
- $\{n > 0\}$
codice che ordina l'array a di n elementi
{sorted(a, n)}
dove: $\text{sorted}(a, n) \equiv (\text{for all } i(1 \leq i < n) \text{ implies } a(i) \leq a(i+1))$

Design by Contract

- *Caratterizzazione logica* delle operazioni (es. metodi, funzioni) e dello stato (es. valori delle variabili, membri di un oggetto) di un programma
- *Precondizioni e postcondizioni* delle operazioni
- *Invarianti*: proprietà dello stato del programma che devono valere in certi passi dell'esecuzione (tipicamente fra un'operazione e l'altra; possono essere violati a operazione in corso)

Esempio

- Invariante per il tipo di dato astratto INSIEME (implementato usando un array IMPL di lunghezza length):

for all i, j ($1 \leq i \leq \text{length}$ and $1 \leq j \leq \text{length}$
and $i \neq j$) **implies** $\text{IMPL}[i] \neq \text{IMPL}[j]$

(cioè non ci sono elementi duplicati)

Esempio

- Precondizione dell'operazione DELETE
exists i ($1 \leq i \leq \text{length}$ **and** $\text{IMPL}[i] = x$)
- Postcondizione:
for all i ($1 \leq i \leq \text{length}$ **implies** $\text{IMPL}[i] \neq x$) **and forall** i ($(1 \leq i \leq \text{old_length} \text{ and } \text{old_IMPL}[i] \neq x) \text{ implies exists } j$ ($1 \leq j \leq \text{length} \text{ and } \text{IMPL}[j] = \text{old_IMPL}[i]$)))

Specifica completa di un tipo di dato astratto

Dati

- INV invariante
- per ogni operazione op_i , pre_i e $post_i$ pre- e post-condizione per op_i

Si impone

- $\{INV \textbf{ and } pre_i\} op_i \{INV \textbf{ and } post_i\}$
- $\{true\}$ costruttore $\{INV\}$

Verifica dinamica (a runtime) di contratti

- Le condizioni sono espresse come espressioni logiche nel linguaggio utilizzato
- Prima dell'esecuzione di un'operazione, si verifica la pre-condizione;
- Dopo l'esecuzione, si verifica la post-condizione
- Vengono verificati anche gli invarianti dopo ogni modifica dello stato interessato (ad es. dopo la chiamata di un metodo di un oggetto)
- In caso di violazioni, viene segnalato errore (ad esempio tramite un'eccezione)
- Alcuni linguaggi o librerie forniscono una sintassi apposita e automatizzano le verifiche
- Altrimenti si possono usare asserzioni (se disponibili) o eccezioni condizionate

Contratti: discussione

L'utilizzo dei contratti deve essere valutato criticamente

- Quanto sono supportati dal linguaggio utilizzato?
- Quali vincoli impongono sulle implementazioni?
 - Caso OOP
- Quanto costano a runtime?
 - Sono disabilitabili in produzione?
- Una violazione in produzione fornisce un caso di test reale
- Conclusione: utili almeno
 - come documentazione formale
 - per rilevare bug nelle operazioni critiche
- Possono essere usati anche per verifica statica (che vedremo):
 - dimostrare che dalla preconditione e dal codice (o da una sua specifica) discende la postcondizione

Specifica di comportamenti che non terminano

- Sistemi reattivi: attendono input e producono output, senza terminare
- Le sequenze di input e output possono non essere finite
- E' sufficiente imporre condizioni
 - sulle sequenze parziali
 - in punti di esecuzione critici
- Esempio: produttore, consumatore, buffer
 - Invariante: `input_sequence = append (output_sequence, contents(buffer))`
 - cioè quanto è stato scritto è la concatenazione di quanto è stato letto e del contenuto del buffer (all'ingresso e all'uscita da tali operazioni)